

Упражнение 17

- Ваши друзья открывают компанию, занимающуюся компьютерной безопасностью, и им необходимо получить лицензии на n разных криптографических продуктов. Согласно действующим требованиям, они могут получать не более одной лицензии в месяц.
- Каждая лицензия в настоящее время стоит \$100, но со временем цена растет по экспоненциальному закону: в частности, лицензия j ежемесячно умножается на коэффициент $r_j > 1$ (где r_j — заданный параметр). Таким образом, если лицензия j приобретается через t месяцев, она будет стоить $100 \cdot r_j^t$. Будем считать, что все скорости роста цены различны, то есть $r_i \neq r_j$ для $i \neq j$ (хотя все они начинаются с одной исходной цены \$100).
- Вопрос: если компания может покупать не более одной лицензии в месяц, в каком порядке следует совершать покупки, чтобы общая потраченная сумма была минимальной?
- Приведите алгоритм, который получает n скоростей роста цены $r_1, r_2, \dots, r_n$ и вычисляет порядок покупки лицензий, чтобы общая сумма потраченных денег была минимальной. Время выполнения алгоритма должно быть полиномиальным по n .

Чтобы выгодно совершать покупки, нужно: отсортировать лицензии по убыванию коэффициентов r_i и покупать в этом порядке

Доказательство оптимального порядка покупки лицензий

Вариант А: $r_i^i + r_{i+1}^{i+1}$

Вариант В: $r_{i+1}^i + r_i^{i+1}$

где i и $i+1$ — это месяцы покупки (временные периоды)

Предположим, что $r_i < r_{i+1}$ (то есть лицензия $i+1$ дорожает быстрее)

Найдём разность:

$$D = (r_i^i + r_{i+1}^{i+1}) - (r_{i+1}^i + r_i^{i+1})$$

Перегруппируем:

$$D = r_i^i - r_{i+1}^i + r_{i+1}^{i+1} - r_i^{i+1}$$

$$D = (r_i^i - r_{i+1}^i) - (r_i^{i+1} - r_{i+1}^{i+1})$$

Обозначим $a = r_i$, $b = r_{i+1}$, где $a < b$:

$$D = (a^i - b^i) - (a^{i+1} - b^{i+1})$$

Функция $f(t) = b^t - a^t$ возрастает по t при $b > a > 1$, потому что:

- b^t растёт быстрее, чем a^t
- Следовательно, $b^{i+1} - a^{i+1} > b^i - a^i$

Отсюда:

$$a^{i+1} - b^{i+1} < a^i - b^i$$

$$a^i + b^{i+1} > a^{i+1} + b^i$$

Вывод через обратную замену

$$r_i^i + r_{i+1}^{i+1} > r_{i+1}^i + r_i^{i+1}$$

Это означает, что выгоднее купить сначала лицензию с БОЛЬШИМ коэффициентом роста r_{i+1} (в месяц i), а потом с меньшим r_i (в месяц $i+1$).

Время работы $O(n \log n)$ на сортировку.

Упражнение 17(2)

Предположим, вместо покупки лицензий с возрастающей ценой вы пытаетесь продавать оборудование, цена которого последовательно снижается.

Цена элемента i снижается с коэффициентом $r_i < 1$ в месяц, начиная с \$100, так что при продаже его через t месяцев Вы получите $100 \cdot r_i^t$.

Иначе говоря, скорость роста теперь не больше, а меньше 1.

Если в месяц продается только один вид оборудования, в каком оптимальном порядке их следует продавать?

Условие:

Цена элемента i снижается с коэффициентом $r_i < 1$ в месяц

Начальная цена: 100

При продаже через t месяцев получаем: $100 \cdot r_i^t$

Рассмотрим два элемента i и j , где $r_i < r_j < 1$.

Нужно сравнить два варианта порядка продажи:

Вариант А: Продаём i первым (месяц 0), затем j (месяц 1)

$$\text{Выручка А} = 100 \cdot r_i^0 + 100 \cdot r_j^1 = 100 + 100 \cdot r_j$$

Вариант В: Продаём j первым (месяц 0), затем i (месяц 1)

$$\text{Выручка В} = 100 \cdot r_j^0 + 100 \cdot r_i^1 = 100 + 100 \cdot r_i$$

Находим разность:

$$\text{Выручка В} - \text{Выручка А} = (100 + 100 \cdot r_i) - (100 + 100 \cdot r_j) = 100 \cdot r_i - 100 \cdot r_j = 100(r_i - r_j)$$

Поскольку $r_i < r_j$, то $r_i - r_j < 0$, следовательно:

$$\text{Выручка В} - \text{Выручка А} < 0$$

Выручка А > Выручка В $\Rightarrow$ выгоднее сначала продать элемент с МЕНЬШИМ коэффициентом r_i . То есть элемент с меньшим r_i (ближе к 0) быстро теряет стоимость и его нужно продать быстрее, а элемент с большим r_j (ближе к 1) медленно теряет стоимость и его можно продать позже

Пример:

Элемент 1: $r_1 = 0.5$ (теряет 50% в месяц)

Элемент 2: $r_2 = 0.9$ (теряет 10% в месяц)

Продаём r_1 первым:

Месяц 1: $100 \cdot 0.5^1 = 50$

Месяц 2: $100 \cdot 0.9^2 = 81$

Итого: 131

Продаём r_2 первым:

Месяц 1: $100 \cdot 0.9^1 = 90$

Месяц 2: $100 \cdot 0.5^2 = 25$

Итого: 115

Упражнение 18

Ваши друзья занялись расширяющейся областью анализа временных рядов. Целью анализа является поиск закономерностей в последовательностях событий, происходящих во времени. Одним из источников таких данных с естественным упорядочением по времени могут служить покупки на биржах (что и когда покупается). Ваши друзья хотят иметь эффективный механизм обнаружения некоторых «закономерностей» в заданной последовательности S таких событий: например, они хотят знать, происходят ли в последовательности S четыре события

купить Yahoo, купить eBay, купить Yahoo, купить Oracle

в такой очередности (но не обязательно подряд).

Они начинают с набора всех возможных событий (например, возможных транзакций) и последовательности S из n таких событий. Заданное событие может встречаться в S несколько раз (например, акции Yahoo могут покупаться многократно в одной последовательности S). Последовательность S' называется подпоследовательностью S , если из S возможно удалить некоторые события так, чтобы оставшиеся события (с соблюдением порядка) были равны последовательности S' . Так, приведенная последовательность из четырех событий является подпоследовательностью для последовательности

купить Amazon, купить Yahoo, купить eBay, купить Yahoo, купить Yahoo, купить Oracle

Ваши друзья хотят вводить короткие последовательности и быстро узнавать, являются ли они подпоследовательностями S . Поставлена следующая задача: предложите алгоритм, который получает две последовательности событий — S' длины m и S длины n (в обеих последовательностях события могут повторяться) и за время $O(m + n)$ решает, является ли S' подпоследовательностью S .

```
def is_subsequence(S_, S):
    m, n = len(S_), len(S)
    i = 0

    for j in range(n):
        if i < m and S_[i] == S[j]: # проходим по S и если нашли совпадение, переходим к следующему элементу S'
            i += 1

    return i == m

S_1 = ["Yahoo", "eBay", "Yahoo", "Oracle"]
S1 = ["Amazon", "Yahoo", "eBay", "Yahoo", "Yahoo", "Oracle"]
print(f"Является ли S_ подпоследовательностью S: {is_subsequence(S_1, S1)}")

S_2 = ["Yahoo", "Oracle", "eBay"]
S2 = ["Yahoo", "eBay", "Amazon", "Yahoo"]
print(f"Является ли S_ подпоследовательностью S: {is_subsequence(S_2, S2)}")
```

Является ли S_ подпоследовательностью S: True
Является ли S_ подпоследовательностью S: False